

# Chapter 3 · Lesson 3 — Progressive Enhancement

## Chapter 3 · Lesson 3 — Progressive Enhancement

*Goal: turn the ugly v1 standings table into the polished version you see in the finished app — by adding one feature at a time. By the end you'll have built <LeagueTable>, <StatCard>, <FormBadge>, and a beautiful, working standings tab.*

### What “progressive enhancement” means here

The single most useful workflow for a React beginner is **add one thing at a time, run the dev server, look at the screen, repeat**. Don't add five features at once and then try to debug the result.

In this lesson we'll add seven features to the standings tab. After each feature you should:

1. Type the change.
2. Save the file (hot reload triggers).
3. Look at the browser.
4. Confirm it looks right.
5. Move on.

If something looks wrong, the diff between the previous-working state and the broken state is small. **Small diffs = easy debugging**.

### v2: extract the table into its own component

Right now the standings logic and the rendering both live in `StandingsTab.tsx`. As we add columns and styling, that file will balloon. Let's pre-emptively split the data computation (which stays in `StandingsTab`) from the rendering (which moves to a new

<LeagueTable> component).

Create components/standings/LeagueTable.tsx:

```
import type { TeamWithScores } from '@/lib/types';
import { th, td } from '@/lib/constants';

interface LeagueTableProps {
  teams: TeamWithScores[];
  onTeamClick?: (team: TeamWithScores) => void;
}

export default function LeagueTable({ teams, onTeamClick }: LeagueTableProps) {
  return (
    <table style={{ width: '100%', borderCollapse: 'collapse' }}>
      <thead>
        <tr style={{ background: '#0F5132', color: '#FBBF24' }}>
          <th style={{ ...th, textAlign: 'center' }}>#</th>
          <th style={th}>Team</th>
          <th style={{ ...th, textAlign: 'center' }}>Total</th>
        </tr>
      </thead>
      <tbody>
        {teams.map((team, i) => (
          <tr key={team.name}
            onClick={() => onTeamClick?.(team)}
            style={{ cursor: onTeamClick ? 'pointer' : 'default' }}>
            <td style={{ ...td, textAlign: 'center', fontWeight: 700 }}>{i + 1}</td>
            <td style={td}>
              <span style={{ marginRight: 8 }}>{team.icon}</span>
              {team.name}
            </td>
            <td style={{ ...td, textAlign: 'center', fontWeight: 800, color: '#FBBF24' }}>
              {team.scores.total}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

```

        </tbody>
      </table>
    );
  }

```

Three new ideas:

### 1. Props for data — teams is now passed in

Before, `StandingsTab` knew about TEAMS directly. Now `<LeagueTable>` is **decoupled** from where the teams come from. It's a pure rendering component: give it teams, it renders them. Anyone could reuse it.

This is the **dumb component pattern** (also called *presentational component*): the component takes data via props and renders it. It doesn't fetch data, doesn't compute scores, doesn't decide what to do on click. It just renders.

### 2. An optional callback prop — `onTeamClick?`

The `?` in `onTeamClick?`: makes the prop optional. The caller can pass it (and clicks become interactive) or not pass it (and clicks do nothing). The component handles both cases:

```
onClick={() => onTeamClick?.(team)}
```

The `?.` is **optional chaining**. If `onTeamClick` is undefined, `onTeamClick?.(team)` does nothing. If it's a function, it gets called with the team.

This is a small but powerful pattern: **make features optional via optional callbacks**. Components become more reusable, and the caller decides what behavior to wire up.

### 3. Style spreading

```
<td style={{ ...td, textAlign: 'center' }}>
```

We import the `td` style object from `lib/constants.ts` and spread it into a new style object, then override `textAlign`. This is JavaScript object spreading — same idea as array spreading. The result is `{ padding: '12px 8px', borderBottom: '1px solid #F3F4F6', fontSize: 15, textAlign: 'center' }`.

The point: **shared base styles + per-cell overrides**, all without CSS classes.

### v3: update StandingsTab to use the new LeagueTable

Update components/tabs/StandingsTab.tsx:

```
import { TEAMS } from '@lib/teams';
import { calculateTeamScores } from '@lib/scoring';
import type { TeamWithScores } from '@lib/types';
import LeagueTable from '../standings/LeagueTable';

interface StandingsTabProps {
  onTeamClick?: (team: TeamWithScores) => void;
}

export default function StandingsTab({ onTeamClick }: StandingsTabProps) {
  const teamsWithScores: TeamWithScores[] = TEAMS.map(t => ({
    ...t,
    scores: calculateTeamScores(t),
  })).sort((a, b) => b.scores.total - a.scores.total);

  return (
    <div>
      <LeagueTable teams={teamsWithScores} onTeamClick={onTeamClick} />
    </div>
  );
}
```

Now StandingsTab does only data prep and delegates rendering. Save and check the browser — should look the same as before, just better-organized internally.

### v4: medal-color the rank circle

The first visual feature we'll add. Change the rank cell in LeagueTable.tsx:

```
<td style={{ ...td, textAlign: 'center', fontWeight: 700 }}>
  {(() => {
```

```

const rank = i + 1;
const rankColor =
  rank === 1 ? '#FBBF24' :    // gold
  rank === 2 ? '#9CA3AF' :    // silver
  rank === 3 ? '#B45309' :    // bronze
  '#E5E7EB';                  // gray
return (
  <div style={{
    width: 32, height: 32,
    borderRadius: '50%',
    background: rankColor,
    color: rank <= 3 ? '#FFFFFF' : '#374151',
    display: 'inline-flex',
    alignItems: 'center',
    justifyContent: 'center',
    fontWeight: 900,
  }}>
    {rank === 1 ? '🥇' : rank}
  </div>
);
})()}}
</td>

```

Whoa — what's that `((() => { ... } ))()` thing?

### IIFE inside JSX (immediately-invoked function expression)

JSX `{}` expects an **expression**, not a sequence of statements. We have several statements (`const rank = ...`, `const rankColor = ...`, then a `return`). To embed those in JSX, we wrap them in an arrow function and immediately invoke it:

```

{(() => {
  // statements here
  return <jsx />;
})()}}

```

That's an IIFE — Immediately-Invoked Function Expression. It evaluates to whatever the

inner function returns.

It's ugly. It works. It's a common React pattern when you have multiple `let/const` statements you want close to where they're used. The cleaner alternative is to **extract this into its own component** — which we'll do in Lesson 5 when it's time to refactor.

For now, the IIFE makes the rank logic local to this cell. **A small ugliness in service of a small feature is fine. Don't pre-refactor.**

### A nested ternary for the colors

```
const rankColor =  
  rank === 1 ? '#FBBF24' :  
  rank === 2 ? '#9CA3AF' :  
  rank === 3 ? '#B45309' :  
  '#E5E7EB';
```

Three checks, four outcomes. Reads top-to-bottom: gold for 1st, silver for 2nd, bronze for 3rd, gray default. **Nested ternaries are fine if formatted vertically and have a clear reading order.** If you find yourself wrapping ternaries on a single line or your editor wraps them weirdly, refactor to a function `getRankColor(rank: number)`.

### `display: 'inline-flex'` for centering

```
display: 'inline-flex',  
alignItems: 'center',  
justifyContent: 'center',
```

This is the canonical CSS recipe for **centering text inside a fixed-size circular badge**. `inline-flex` lays out the children with flexbox; `alignItems: center` is vertical centering; `justifyContent: center` is horizontal. Works for any container with a defined size.

Save. Refresh. The leader has a 🏆 in a gold circle, 2nd has silver, 3rd has bronze. It's starting to look real.

## v5: per-round columns

Add columns for R1, R2, QF, SF, Final. Update the header and body of `LeagueTable.tsx`:

```
<thead>
  <tr style={{ background: '#0F5132', color: '#FBBF24' }}>
    <th style={{ ...th, textAlign: 'center' }}>#</th>
    <th style={th}>Team</th>
    <th style={{ ...th, textAlign: 'center' }}>R1</th>
    <th style={{ ...th, textAlign: 'center' }}>R2</th>
    <th style={{ ...th, textAlign: 'center' }}>QF</th>
    <th style={{ ...th, textAlign: 'center' }}>SF</th>
    <th style={{ ...th, textAlign: 'center' }}>Final</th>
    <th style={{ ...th, textAlign: 'center' }}>Total</th>
  </tr>
</thead>
```

And in the body, after the team-name cell:

```
<td style={{ ...td, textAlign: 'center' }}>
  <div style={{ fontWeight: 700, fontSize: 17, color: '#0F5132' }}>
    {team.scores.r1}
  </div>
  <div style={{ fontSize: 11, color: '#6B7280' }}>/ 48</div>
</td>
<td style={{ ...td, textAlign: 'center' }}>
  <div style={{ fontWeight: 700, fontSize: 17, color: '#0F5132' }}>
    {team.scores.r2}
  </div>
  <div style={{ fontSize: 11, color: '#6B7280' }}>/ 24</div>
</td>
{ /* ...QF (/12), SF (/6), Final (/3)... */ }
```

The “/ 48” “/ 24” lines show the **maximum possible score for the round**, computed at design time:

- R1: 16 matches × 3 points = 48
- R2: 8 × 3 = 24
- QF: 4 × 3 = 12

- SF:  $2 \times 3 = 6$
- Final:  $1 \times 3 = 3$

Hard-coded because they're constants of the tournament structure. If we ever changed the bracket size, we'd want to compute these from `ROUND1_MATCHES.length * 3` etc.

Save. Refresh. The table now has 8 columns and feels like a real fantasy leaderboard.

## v6: the Final Pick chip

Add another column showing each team's Final Pick:

```
<th style={th}>Final Pick</th>

<td style={{ ...td, textAlign: 'center' }}>
  <div style={{
    display: 'inline-block',
    padding: '4px 10px',
    borderRadius: 12,
    background: team.final === 'Wu Yize' ? '#DCFCE7' : '#FEE2E2',
    color: team.final === 'Wu Yize' ? '#166534' : '#991B1B',
    fontWeight: 700,
    fontSize: 12,
  }}>
    {team.final}
  </div>
</td>
```

A pill-shaped badge with a 🏆 and the team's pick.

## The conditional coloring

```
background: team.final === 'Wu Yize' ? '#DCFCE7' : '#FEE2E2',
color:      team.final === 'Wu Yize' ? '#166534' : '#991B1B',
```

If the team picked the actual champion (Wu Yize), the chip is **green**. Otherwise it's **red**. This is the post-tournament color rule we discussed in the original course's Lesson 7 — hard-coding the champion's name into the styling logic.



A more general version would compare against `FINAL_MATCH[0]?.winner`. The hard-coded version is faster to write but ties this code to the 2026 tournament. **For a single-tournament app, fine. For a reusable framework, not fine.** Most real codebases have a mix.

## v7: the FormBadge

Last column: a “Form” indicator showing whether the team is improving or slipping between rounds.

Create `components/standings/FormBadge.tsx`:

```
import { TrendingUp, TrendingDown, Minus } from 'lucide-react';

interface FormBadgeProps {
  r1Pct: number;
  r2Pct: number;
}

export default function FormBadge({ r1Pct, r2Pct }: FormBadgeProps) {
  const trending = r2Pct > r1Pct ? 'up' : r2Pct < r1Pct ? 'down' : 'flat';
  const Icon = trending === 'up' ? TrendingUp : trending === 'down' ? TrendingDown : TrendingFlat;
  const color = trending === 'up' ? '#16A34A' : trending === 'down' ? '#DC2626' : 'gray';
  const label = trending === 'up' ? 'Rising' : trending === 'down' ? 'Falling' : 'Flat';

  return (
    <div style={{
      display: 'inline-flex',
      alignItems: 'center',
      gap: 4,
      color,
      fontSize: 13,
      fontWeight: 700,
    }}>
      <Icon size={16} strokeWidth={3} />
      {label}
    </div>
  );
}
```

```
);
}
```

Then in `LeagueTable.tsx`:

```
import FormBadge from './FormBadge';

// ...in the row:
<td style={{ ...td, textAlign: 'center' }}>
  <FormBadge
    r1Pct={(team.scores.r1 / 48) * 100}
    r2Pct={(team.scores.r2 / 24) * 100}
  />
</td>
```

Three new ideas:

### Lucide icons as components

```
import { TrendingUp, TrendingDown, Minus } from 'lucide-react';
```

Lucide is a tree-shakeable icon library. We import three icon components and treat them like any other React component (`<TrendingUp size={16} />`).

The **tree-shakeable** part matters: the production bundle only includes the icons you actually import. If we import 3 icons, we ship 3, not all ~1,000 in the library.

### Picking a component dynamically

```
const Icon = trending === 'up' ? TrendingUp : trending === 'down' ? TrendingDown : TrendingDown
// ...
<Icon size={16} strokeWidth={3} />
```

`Icon` is a variable holding **a component**. We then use it as a JSX tag. This works because JSX is just a function call; `<Icon />` compiles to `React.createElement(Icon, ...)` and the `Icon` reference can come from anywhere — a prop, a variable, a `useState` value, anything.

Recall the earlier rule: **JSX tag names must be capitalized to be treated as components**.

We renamed our local variable `Icon` (capital I) so JSX recognizes it. If we'd used `icon`, JSX would have looked for an HTML `<icon>` tag.

### Calculating percentages inline

```
r1Pct= {(team.scores.r1 / 48) * 100}
```

We pass percentages, not raw scores. Why? Because R1 has a max of 48 and R2 has a max of 24 — direct comparison would be misleading. Comparing **percentages** of perfect normalizes across rounds.

Save. Refresh. Each row now has a Form indicator showing whether they got better or worse from R1 to R2. Most teams will probably show “Rising” or “Falling” since percentages rarely tie exactly.

### v8: stat cards above the table

The top of the standings tab in the finished app shows three stat tiles: **Leader**, **Total Matches**, **Avg Score**. Create components/standings/StatCard.tsx (this is the file we walked through in Lesson 1):

```
import type { LucideIcon } from 'lucide-react';

interface StatCardProps {
  icon: LucideIcon;
  label: string;
  value: string | number;
  color: string;
  bgColor: string;
}

export default function StatCard({ icon: Icon, label, value, color, bgColor }: StatCardProps) {
  return (
    <div style={{
      background: `linear-gradient(135deg, ${bgColor} 0%, ${bgColor}DD 100%)`,
      color: '#FFFFFF',
      padding: 24,
```

```

    borderRadius: 16,
    boxShadow: `0 8px 24px ${bgColor}40`,
  }}>
    <Icon size={28} strokeWidth={2.5} />
    <div style={{ fontSize: 13, letterSpacing: '1.5px', fontWeight: 600, opacity: 0.8 }}>
      {label}
    </div>
    <div style={{ fontSize: 32, fontWeight: 900, fontFamily: "'Roboto Slab', serif" }}>
      {value}
    </div>
  </div>
);
}

```

Then update `StandingsTab.tsx` to render three cards above the table:

```

import { Trophy, Calendar, Target } from 'lucide-react';
import StatCard from '../standings/StatCard';
import { ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCHES } from '../constants';

export default function StandingsTab({ onTeamClick }: StandingsTabProps) {
  const teamsWithScores: TeamWithScores[] = TEAMS.map(t => ({
    ...t,
    scores: calculateTeamScores(t),
  })).sort((a, b) => b.scores.total - a.scores.total);

  const totalMatches =
    ROUND1_MATCHES.length + ROUND2_MATCHES.length +
    QF_MATCHES.length + SF_MATCHES.length + FINAL_MATCHES.length;

  const leader = teamsWithScores[0];
  const avgScore = Math.round(
    teamsWithScores.reduce((sum, t) => sum + t.scores.total, 0) / teamsWithScores.length
  );

  return (

```

```

<div>
  <div style={{
    display: 'grid',
    gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))',
    gap: 16,
    marginBottom: 24,
  }}>
    <StatCard icon={Trophy} label="LEADER" value={leader.name} col
    <StatCard icon={Calendar} label="MATCHES" value={totalMatches} col
    <StatCard icon={Target} label="AVG SCORE" value={avgScore} col
  </div>

  <LeagueTable teams={teamsWithScores} onTeamClick={onTeamClick} />
</div>
);
}

```

Three new things to call out:

### CSS Grid with auto-fit and minmax

```
gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))',
```

This is the **modern way to do responsive card grids**. It says:

- Repeat as many columns as fit.
- Each column is **at least 220px** and **at most 1 fraction** of available space.

So if the screen is 660px wide, you get 3 columns of ~220px. If it's 480px, you get 2 columns. If it's 220px, you get 1. **Without media queries**. This single CSS line replaces what used to take a dozen breakpoints.

Memorize this pattern. It's interview gold.

### Computed values inside the component

```

const totalMatches = ROUND1_MATCHES.length + ROUND2_MATCHES.length + ...;
const leader = teamsWithScores[0];

```

```
const avgScore = Math.round(teamsWithScores.reduce((sum, t) => sum + t.scores
```

We derive three values from the data and pass them as props to the StatCards. **Derived state stays close to where it's used** — no need to compute these elsewhere.

The `.reduce((sum, t) => sum + t.scores.total, 0)` is JavaScript's "sum an array" idiom. `0` is the initial value of `sum`. For each team `t`, we add `t.scores.total` to `sum`. End result: total points across all teams. Divide by 8 to get the average. `Math.round` for clean integer display.

### Importing icons by name

```
import { Trophy, Calendar, Target } from 'lucide-react';
```

Three named imports. Each is a React component. We pass them as props to `<StatCard>`, which renders them.

Notice we **don't** instantiate them — we pass the component itself, not `<Trophy />`. The `StatCard` component receives the component-class and renders it inside its own JSX. This is how you build flexible, "give me an icon to render here" APIs.

### What v8 looks like

Save everything and refresh. You should now see:

- Three colored stat cards at the top: green "LEADER" card with the leader's name, red "MATCHES" card showing 31, purple "AVG SCORE" card.
- Below them, the full standings table with medal-color rank badges, per-round columns, Final Pick chips, Form badges.

It looks like a real fantasy app now.

### What we did NOT do (yet)

Notice we still don't have:

- The hero header (gradient banner).
- The tab nav bar.
- Multiple tabs (Matches, Predictions, Players, Analytics).

Those are Chapter 4. Right now we have **one tab, fully built, with no tab bar**. That's intentional — building one screen completely before adding navigation lets you focus on each problem in turn.

## The discipline you just practiced

Read back through this lesson. We added **eight versions** of the same component, each one a small addition:

- v1: plain table (Lesson 2)
- v2: extract LeagueTable
- v3: wire it up
- v4: medal-color rank
- v5: per-round columns
- v6: Final Pick chip
- v7: FormBadge
- v8: stat cards

Each version was 5–30 lines of new code. Each one was independently verifiable in the browser. **No version did three things at once.**

This is **the way professional React engineers build features**. Not all in one go — incrementally. The git history of any well-managed React project looks exactly like this: dozens of small commits, each adding one tiny thing. The big-bang “I built it all in one PR” approach is what produces unreviewable code and untraceable bugs.

**Interview tip.** When asked “*how do you approach building a new feature?*”, your answer should sound like “*I start with the smallest version that proves the data path works, then I add visual polish one piece at a time, running the dev server and looking at the screen between every change. Small commits, easy to review, easy to roll back.*” That answer signals you’ve shipped real software.

## Vibe prompt you would have used (per feature)

Feature by feature, the prompts would have been:

v4: “*In the rank cell of <LeagueTable>, replace the plain number with a 32x32 circle. Background: gold for rank 1, silver for 2, bronze for 3, gray otherwise. Replace the number with a 🏆 emoji for rank 1.*”

v5: “Add R1, R2, QF, SF, Final columns to `<LeagueTable>`, between Team and Total. Show the round score as a big number with `/ {maxScore}` underneath in small gray text. Maxes: 48, 24, 12, 6, 3.”

v6: “Add a Final Pick column to `<LeagueTable>`. Show `team.final` inside a rounded chip with a 🏆 emoji. Background green if `team.final === 'Wu Yize'`, red otherwise.”

v7: “Create `<FormBadge>` taking `r1Pct` and `r2Pct` numbers. Show TrendingUp icon + ‘Rising’ (green) if `r2 > r1`, TrendingDown + ‘Falling’ (red) if `r2 < r1`, Minus + ‘Steady’ (gray) otherwise. Use Lucide icons. Then add a Form column to `<LeagueTable>` using it.”

Each prompt is **one feature, one component, one paragraph**. That’s the right grain. Compare to a bad prompt: “*make the standings table nicer*” — that produces an unmaintainable wall of CSS that you can’t reason about.

## CHECK YOURSELF

1. **The IIFE inside the rank cell `((() => { ... } ))()` works but is ugly. What’s the cleaner refactor?** What’s the cost of doing it now vs leaving it?
2. **`onTeamClick?: (team) => void` is optional. What does the `?.` in `onTeamClick?.(team)` do?** What would happen without the `?` if a parent didn’t pass `onTeamClick?`?
3. **`gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))'` — explain in plain English what this does.** What replaces this in old-style CSS?
4. **`<StatCard icon={Trophy} />` passes the Trophy *component itself*, not `<Trophy />`. Why?** What would go wrong if you wrote `icon={<Trophy />}`?
5. **The Final Pick chip hard-codes ‘Wu Yize’ for its color choice.** Write the one-line edit that makes it tournament-agnostic by reading from `FINAL_MATCH[0]?.winner` instead.

Once you’ve answered (and ideally typed #5), you’ve completed Chapter 3. The standings tab is fully built. **Onward to Chapter 4 — State & Hooks**, where we add the tab bar and finally meet `useState`.